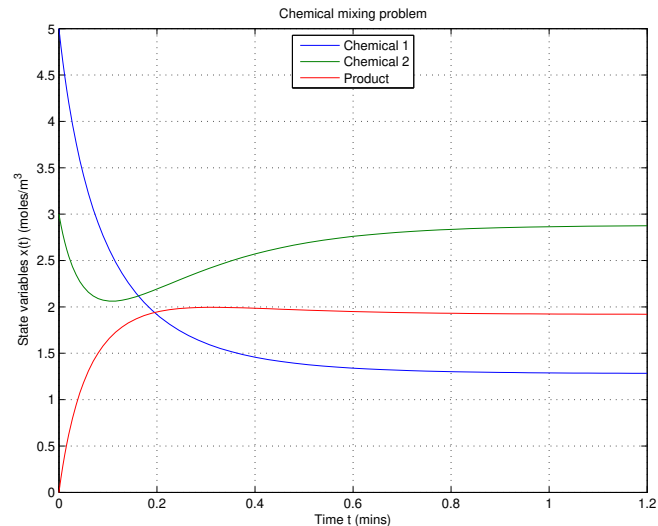


§6 Ordinary Differential Equations (ODEs)

Motivation.

- 🔴 *Chemical mixing problem*: Given a model of a stirred tank chemical reactor in which two chemicals react under controlled conditions to produce a certain product, find the concentrations of the two chemicals and the product over a period of time.

- 🟡 Need to solve a *system of first order ODEs*
- 🟡 Numerical solution by time discretization



Key concepts.

- 🔴 First order *Initial Value Problem* (IVP): find $y(t)$ given $y' = f(t, y)$ and y_0
 - 🟡 *Euler's method*: $y_{n+1} = y_n + hf(t_n, y_n)$
 - 🟡 System of first order ODEs
 - 🟡 Higher order ODEs
 - 🟡 Implicit v.s. explicit methods
 - 🟡 One-step v.s. multi-step methods
 - 🟡 Predictor-corrector methods
 - 🟡 Error, step size control, stiff problems
- 🔴 Two-point *Boundary Value Problems* (BVP): find $y(t)$ on $t \in [a, b]$ given $y'' = f(t, y, y')$, $y(a) = y_a$ and $y(b) = y_b$
 - 🟡 Shooting methods

References. Rao 9.1 – 9.13, 10.1 – 10.3

Accompanying files. `euler.m` `heun.m` `rk4.m` `math2089ode.py`

§6A. Initial Value Problem and Euler's Method.

- Differential Equations – function defined by equations involving derivatives
- *Ordinary Differential Equation* (ODE)
 - Function of **one** variable $y(t)$
 - Variable t often represents *time*
 - e.g. concentration $C(t)$ of a chemical changing over time t
- *Partial Differential Equation* (PDE)
 - Function of **more than one** variable, e.g. $u(x, y)$, $u(x, t)$, $u(x, y, z)$
 - Variables can represent *space* (x, y, z) and *time* t
 - e.g. temperature $u(x, y, t)$ at position (x, y) in a plate at time t
- The *order* of an ODE or PDE is the highest derivative present
- Domain for variables: e.g. $t \in [0, T]$, $x \in [a, b]$, $(x, y) \in \mathcal{D}$
- *Initial conditions* – values at starting time, e.g. $y(0) = 1$
- *Boundary conditions* – values at boundary of space or time domain, e.g. on the domain $t \in [0, 1]$ specify $y(0) = 1$ and $y(1) = 2$

Example. *Newton's law of cooling*: The rate of which a body cools is proportional to the difference between the temperature of the body and the temperature of the surrounding medium.

- Let $U(t)$ denote temperature of body at time t
- First order ODE: $\frac{dU}{dt} = k(U - u_M)$
 - Temperature of surrounding medium $= u_M$
 - Rate constant $k < 0$ for cooling
- Initial condition: e.g. $U(0) = 37^\circ\text{C}$ for human body

Example. *Vertical motion under gravity*: A free falling object close to the surface of the Earth accelerates at a constant rate $g = 9.8\text{m/s}^2$.

- Let $s(t)$ be vertical distance body travels (with upwards positive)
- Time domain $[0, T]$, T = time body hits the ground
- Second order ODE: $\frac{d^2s}{dt^2} = -g$
- Starting height and initial velocity: e.g. $s(0) = 50\text{m}$ and $s'(0) = 0\text{m/s}$

● First order *Initial Value Problem* (IVP)

$$\frac{dy}{dt} = f(t, y) \quad \text{for } t \in [t_0, t_{\max}], \quad \text{with } y(t_0) = y_0$$

- Given function $f(t, y)$
- Given initial time t_0 (not necessarily 0)
- Given *initial value* y_0
- Unknown function $y(t)$ to be found
- Explicit dependence of y on t suppressed. Formally we should write

$$\frac{dy(t)}{dt} = f(t, y(t))$$

- Often we simply write $y' = f(t, y)$
- **Theorem** (*existence and uniqueness of solution*). If $f(t, y)$ and $\frac{\partial f(t, y)}{\partial y}$ are continuous and bounded for all $t \in [t_0, t_{\max}]$ and $y \in \mathbb{R}$, then the IVP has a unique solution in the time interval $[t_0, t_{\max}]$.
(Solution can still be unknown.)

Exercise. For each IVP below, verify that the given solution satisfies the differential equation and the initial condition.

● IVP: $y' = 1 + \frac{y}{t}$, $t \in [1, 2]$, $y(1) = 2$

Solution: $y(t) = t \ln(t) + 2t$

● IVP: $y' = \frac{2}{t}y + t^2 e^t$, $t \in [1, 2]$, $y(1) = 0$

Solution: $y(t) = t^2 (e^t - e)$

● Common setup of numerical methods for solving first order IVPs

- *Discretization* of time interval $[t_0, t_{\max}]$

$$t_0 < t_1 < t_2 < \cdots < t_N = t_{\max}$$

- Equally-spaced grid points

$$t_n = t_0 + nh, \quad n = 0, 1, \dots, N, \quad h = \frac{t_{\max} - t_0}{N}$$

- h is known as the *step size*

- Denote the true solution by $y(t)$

- Denote the approximation of $y(t_n)$ by y_n

- Obtain approximation at the grid points in the order $y_1, y_2, y_3, \dots$

- Denote the error at t_n by $E_n := y(t_n) - y_n$

- Obtain the approximation at non-grid points by interpolation

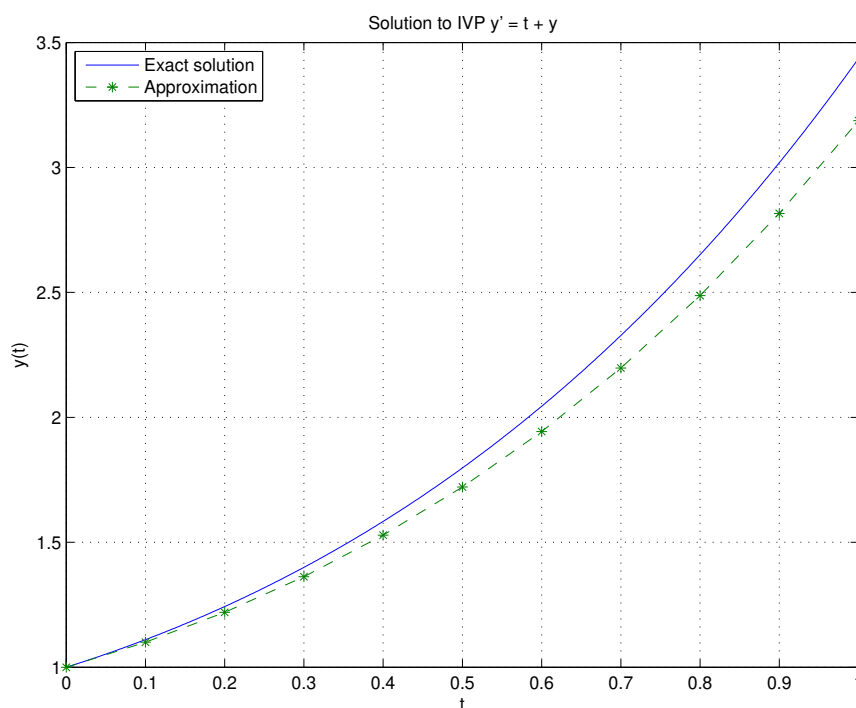
- *Euler's method*: solve $y' = f(t, y)$, $t \in [t_0, t_{\max}]$, $y(t_0) = y_0$ by

$$y_{n+1} = y_n + h f(t_n, y_n), \quad n = 0, 1, \dots, N-1$$

Example. Use Euler's method with step size $h = 0.1$ to solve the IVP

$$y' = t + y, \quad t \in [0, 1], \quad y(0) = 1.$$

Compare with the exact solution $y(t) = 2e^t - 1 - t$.



n	t_n	$y(t_n)$	y_n	E_n
0	0.00	1.0000	1.0000	0
1	0.10	1.1103	1.1000	1.03×10^{-2}
2	0.20	1.2428	1.2200	2.28×10^{-2}
3	0.30	1.3997	1.3620	3.77×10^{-2}
4	0.40	1.5836	1.5282	5.54×10^{-2}
5	0.50	1.7974	1.7210	7.64×10^{-2}
	$\vdots$			

Answer.

(i) *Hand calculation:*

Euler's method for this IVP is $y_{n+1} = y_n + h f(t_n, y_n) = y_n + 0.1(t_n + y_n)$

$$t_0 = 0 \quad y_0 = 1$$

$$t_1 = 0.1 \quad y_1 = y_0 + 0.1(t_0 + y_0) = 1 + 0.1(0 + 1) = 1.1$$

$$t_2 = 0.2 \quad y_2 = y_1 + 0.1(t_1 + y_1) = 1.1 + 0.1(0.1 + 1.1) = 1.22$$

$$t_3 = 0.3 \quad y_3 = y_2 + 0.1(t_2 + y_2) = 1.22 + 0.1(0.2 + 1.22) = 1.362$$

(ii) *Step-by-step calculation using MATLAB :*

```
f = @(t, y) t + y
h = 0.1
t0 = 0, y0 = 1

t1 = t0 + h, y1 = y0 + h * f(t0, y0)
t2 = t1 + h, y2 = y1 + h * f(t1, y1)
t3 = t2 + h, y3 = y2 + h * f(t2, y2)
```

(iii) *Step-by-step calculation using Python :*

```
def f(t, y): return t + y
h = 0.1
t0 = 0; y0 = 1

t1 = t0 + h; y1 = y0 + h * f(t0, y0)
t2 = t1 + h; y2 = y1 + h * f(t1, y1)
t3 = t2 + h; y3 = y2 + h * f(t2, y2)

print('t0 =', t0, ' y0 =', y0)
print('t1 =', t1, ' y1 =', y1)
print('t2 =', t2, ' y2 =', y2)
print('t3 =', t3, ' y3 =', y3)
```

(iv) *Solution using MATLAB with a loop:*

```
f = @(t, y) t + y;
t0 = 0; tmax = 1; N = 10; h = (tmax - t0) / N;
y0 = 1;

t = [t0 : h : tmax];          % t is a row vector of time steps
y = zeros(1,N+1);            % y is a row vector of solutions

y(1) = y0;
for n = 1:N
    y(n+1) = y(n) + h * f(t(n), y(n)); % Euler's method
end

[t; y]'                      % transpose to display table
```

(v) *Solution using Python with a loop:*

```
from numpy import arange, zeros, vstack

def f(t, y): return t + y
t0 = 0; tmax = 1; N = 10; h = (tmax - t0) / N
y0 = 1

t = arange(t0, tmax + h/2, h) # t is an array of time steps
y = zeros(N+1)                # y is an array of solutions

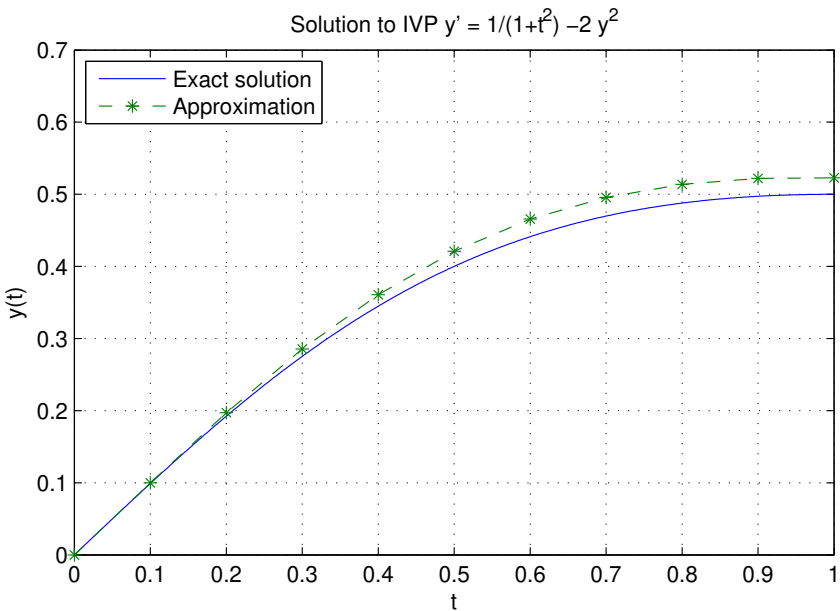
y[0] = y0
for n in range(N):
    y[n+1] = y[n] + h * f(t[n], y[n]) # Euler's method

vstack((t, y)).T              # transpose to display table
```

Exercise. Use Euler’s method with step size $h = 0.1$ to solve the IVP

$$y' = \frac{1}{1+t^2} - 2y^2, \quad t \in [0,1], \quad y(0) = 0.$$

Compare with the exact solution $y(t) = \frac{t}{1+t^2}$.



n	t_n	$y(t_n)$	y_n	E_n
0	0.00	0.0000	0.0000	0
1	0.10	0.0990	0.1000	-9.90×10^{-4}
2	0.20	0.1923	0.1970	-4.70×10^{-3}
3	0.30	0.2752	0.2854	-1.02×10^{-2}
4	0.40	0.3448	0.3609	-1.60×10^{-2}
5	0.50	0.4000	0.4210	-2.10×10^{-2}
	$\vdots$			

Adapt the previous MATLAB and Python codes to solve this problem.

§6B. Systems of First Order ODEs and Higher Order ODEs.

● System of first order ODEs

$$\begin{cases} \frac{dx_1}{dt} = f_1(t, x_1, \dots, x_m) \\ \frac{dx_2}{dt} = f_2(t, x_1, \dots, x_m) \\ \vdots \\ \frac{dx_m}{dt} = f_m(t, x_1, \dots, x_m) \end{cases} \quad \text{or simply} \quad \begin{cases} x'_1 = f_1(t, x_1, \dots, x_m) \\ x'_2 = f_2(t, x_1, \dots, x_m) \\ \vdots \\ x'_m = f_m(t, x_1, \dots, x_m) \end{cases}$$

● In vector form

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(t, \mathbf{x}) \quad \text{or simply} \quad \mathbf{x}' = \mathbf{f}(t, \mathbf{x})$$

● Given m functions $f_1(t, x_1, \dots, x_m), \dots, f_m(t, x_1, \dots, x_m)$

● Given initial time t_0

● Given a **vector** of *initial values* $\mathbf{x}(t_0) = \mathbf{x}_0$

● Unknown **vector-valued** function $\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ \vdots \\ x_m(t) \end{bmatrix}$ to be found

● *Euler's method* $\mathbf{x}_{n+1} = \mathbf{x}_n + h \mathbf{f}(t_n, \mathbf{x}_n)$

Example. Here is an example of a system of two first order ODEs ($m = 2$)

$$\begin{cases} x'_1 = x_2, \\ x'_2 = 2x_1 t + x_2, \end{cases} \quad t \in [1, 1.4], \quad \begin{cases} x_1(1) = 2, \\ x_2(1) = 1, \end{cases}$$

or written in vector form,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{x}' = \begin{bmatrix} x_2 \\ 2x_1 t + x_2 \end{bmatrix}, \quad t \in [1, 1.4], \quad \mathbf{x}_0 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Example. MATLAB `euler.m` (download from Moodle) provides a function to solve a system of first order ODEs by Euler's method:

```
function [t, y] = euler(f, t0, tmax, N, y0)
% explain input and output...

y0 = y0(:);           % ensure that y0 is a column vector
m = length(y0);       % m is the dimension of the system
h = (tmax - t0) / N;   % h is the step size
t = [t0 : h : tmax];   % t is a vector of size 1 by N+1
y = zeros(m, N+1);     % y is a matrix of size m by N+1

y(:,1) = y0;           % y0 is a column vector of size m by 1
for n = 1:N             % f returns a column vector of size m by 1
    y(:,n+1) = y(:,n) + h * f(t(n), y(:,n)); % Euler's method
end

% transpose to match the output format of MATLAB function ode45
t = t'; % output t is a vector of size N+1 by 1
y = y'; % output y is a matrix of size N+1 by m
```

Python `math2089ode.py` (download from Moodle) provides an equivalent function:

```
def euler(f, t0, tmax, N, y0):
    """ Explain input and output... """

    m = size(y0)           # m is the dimension of the system
    h = (tmax - t0) / N     # h is the step size
    t = arange(t0, tmax + h/2, h) # t is an array of size N+1
    y = zeros((N+1, m))     # y is a matrix of size N+1 by m

    y[0,:] = y0             # y0 is an array of size N+1
    for n in range(N):      # f returns an array of size m
        y[n+1,:] = y[n,:] + h * f(t[n], y[n,:]) # Euler's method

    return t, y
```

Notes:

- (i) In MATLAB you need to have the file `euler.m` in your working directory.
- (ii) In Python you can either copy and paste the function into your current script, or have the file `math2089ode.py` in your working directory and then

```
from math2089ode import euler
```

- (iii) For convenience, `euler.m` works internally with the transpose of `t` and `y`.

Example. *Chemical mixing problem:* Given a model of a stirred tank chemical reactor in which two chemicals react under controlled conditions to produce a certain product, find the concentrations of the two chemicals and the product over a period of time.

● State variables

● x_1 = concentration of chemical 1 (moles/m³)

● x_2 = concentration of chemical 2 (moles/m³)

● x_3 = concentration of product (moles/m³)

● Governing system of ODEs

$$\begin{cases} V \frac{dx_1}{dt} - q u_1 + q x_1 + V r = 0 \\ V \frac{dx_2}{dt} - q u_2 + q x_2 + V r = 0 \\ V \frac{dx_3}{dt} + q x_3 - V r = 0 \end{cases} \Rightarrow \begin{bmatrix} \frac{dx_1}{dt} \\ \frac{dx_2}{dt} \\ \frac{dx_3}{dt} \end{bmatrix} = \begin{bmatrix} \frac{q}{V} u_1 - \frac{q}{V} x_1 - r \\ \frac{q}{V} u_2 - \frac{q}{V} x_2 - r \\ -\frac{q}{V} x_3 + r \end{bmatrix}$$

where

● V = volume of the tank (m³) = 2

● q = constant flow rate into and out of the tank (m³/min) = 10

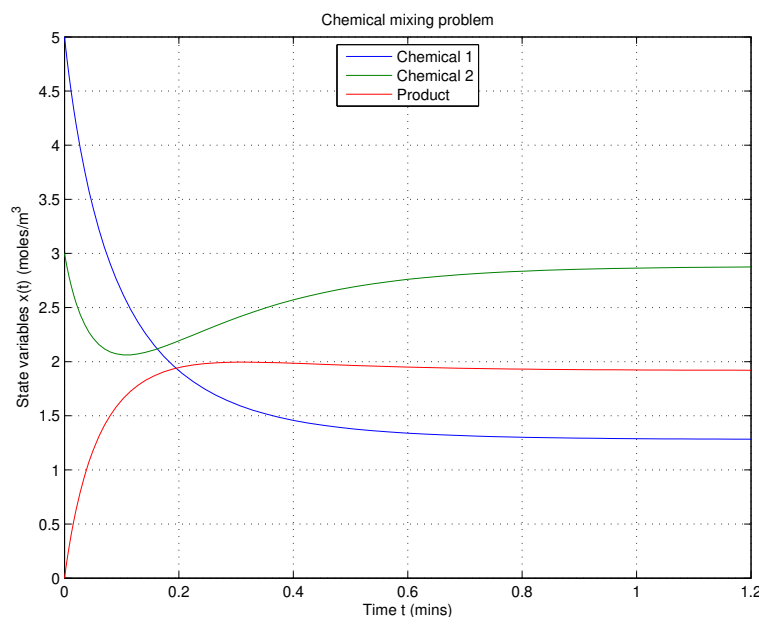
● u_1 = concentration (moles/m³) of chemical 1 in the input stream = 3.2

● u_2 = concentration (moles/m³) of chemical 2 in the input stream = 4.8

● r = reaction rate (moles/m³-min) = $2.6x_1x_2$

● Initial conditions: $x_1(0) = 5$, $x_2(0) = 3$, $x_3(0) = 0$

● Interested in the values of $x_1(t)$, $x_2(t)$, $x_3(t)$ at $t = 72$ seconds (1.2 min)



In MATLAB we do

```
f = @(t, x) [16 - 5 * x(1) - 2.6 * x(1) * x(2);  
             24 - 5 * x(2) - 2.6 * x(1) * x(2);  
             - 5 * x(3) + 2.6 * x(1) * x(2)];  
  
t0 = 0; tmax = 1.2; N = 100;  
x0 = [5; 3; 0];  
  
[t, y] = euler(f, t0, tmax, N, x0); % t is a vector of size 101 by 1  
                                     % y is a matrix of size 101 by 3  
plot(t, y)                           % plot 3 chemicals against time  
grid on  
legend('Chemical 1', 'Chemical 2', 'Product')
```

In Python we do

```
from numpy import array  
from math2089ode import euler  
import matplotlib.pyplot as plt  
  
def f(t, x): return array([16 - 5 * x[0] - 2.6 * x[0] * x[1],  
                           24 - 5 * x[1] - 2.6 * x[0] * x[1],  
                           - 5 * x[2] + 2.6 * x[0] * x[1]])  
  
t0 = 0; tmax = 1.2; N = 100  
x0 = array([5, 3, 0])  
  
t, y = euler(f, t0, tmax, N, x0) # t is an array of size 101  
                                 # y is a matrix of size 101 by 3  
plt.plot(t, y)                  # plot 3 chemicals against time  
plt.grid(True)  
plt.legend(('Chemical 1', 'Chemical 2', 'Product'))  
plt.show()
```

Exercise. Return to solve the problem on Page 6 using MATLAB `euler.m` and Python `euler` function.

- A m th order nonlinear ODE is of the form

$$\frac{d^m y}{dt^m} = f\left(t, y, \frac{dy}{dt}, \frac{d^2 y}{dt^2}, \dots, \frac{d^{m-1} y}{dt^{m-1}}\right)$$

- Often we simply write $y^{(m)} = f(t, y, y', y'', \dots, y^{(m-1)})$
- Can be converted into a system of m first order ODEs as follows:

- Let

$$\begin{cases} x_1 = y \\ x_2 = y' \\ x_3 = y'' \\ \vdots \\ x_m = y^{(m-1)} \end{cases}$$

- Then

$$\begin{cases} x_1' = y' = x_2 \\ x_2' = y'' = x_3 \\ x_3' = y''' = x_4 \\ \vdots \\ x_m' = y^{(m)} = f(t, y, y', y'', \dots, y^{(m-1)}) = f(t, x_1, x_2, x_3, \dots, x_m) \end{cases}$$

- Thus we obtain

$$\mathbf{x}' = \mathbf{f}(t, \mathbf{x}),$$

with

$$\mathbf{x} = \begin{bmatrix} y \\ y' \\ y'' \\ \vdots \\ y^{(m-1)} \end{bmatrix} \quad \text{and} \quad \mathbf{f}(t, \mathbf{x}) = \begin{bmatrix} x_2 \\ x_3 \\ x_4 \\ \vdots \\ f(t, x_1, x_2, x_3, \dots, x_m) \end{bmatrix}$$

- *Euler's method* $\mathbf{x}_{n+1} = \mathbf{x}_n + h \mathbf{f}(t_n, \mathbf{x}_n)$

Example. Convert the *vertical motion under gravity* example

$$y'' = -9.8, \quad y(0) = 50, \quad y'(0) = 0$$

into a system of first order ODEs. Use Euler's method with step size $h = 0.1$ to estimate $y(0.2)$, i.e., the height after 0.2 seconds.

Answer.

(i) Let

$$\begin{cases} x_1 = y \\ x_2 = y' \end{cases} \implies \begin{cases} x'_1 = y' = x_2 \\ x'_2 = y'' = -9.8 \end{cases}$$

Thus we obtain $\mathbf{x}' = \mathbf{f}(t, \mathbf{x})$, with

$$\mathbf{x} = \begin{bmatrix} y \\ y' \end{bmatrix}, \quad \mathbf{f}(t, \mathbf{x}) = \begin{bmatrix} x_2 \\ -9.8 \end{bmatrix},$$

and the initial condition is

$$\mathbf{x}_0 = \begin{bmatrix} y(0) \\ y'(0) \end{bmatrix} = \begin{bmatrix} 50 \\ 0 \end{bmatrix}.$$

(ii) Euler's method is

$$\mathbf{x}_{n+1} = \mathbf{x}_n + h \mathbf{f}(t_n, \mathbf{x}_n) = \begin{bmatrix} x_{n,1} \\ x_{n,2} \end{bmatrix} + h \begin{bmatrix} x_{n,2} \\ -9.8 \end{bmatrix}.$$

Hand calculation of two steps with step size $h = 0.1$:

$$t_0 = 0 \quad \mathbf{x}_0 = \begin{bmatrix} 50 \\ 0 \end{bmatrix} \quad f(t_0, \mathbf{x}_0) = \begin{bmatrix} x_{0,2} \\ -9.8 \end{bmatrix} = \begin{bmatrix} 0 \\ -9.8 \end{bmatrix}$$

$$\begin{aligned} t_1 = 0.1 \quad \mathbf{x}_1 &= \mathbf{x}_0 + h f(t_0, \mathbf{x}_0) & f(t_1, \mathbf{x}_1) &= \begin{bmatrix} x_{1,2} \\ -9.8 \end{bmatrix} = \begin{bmatrix} -0.98 \\ -9.8 \end{bmatrix} \\ &= \begin{bmatrix} 50 \\ 0 \end{bmatrix} + 0.1 \begin{bmatrix} 0 \\ -9.8 \end{bmatrix} \\ &= \begin{bmatrix} 50 \\ -0.98 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} t_2 = 0.2 \quad \mathbf{x}_2 &= \mathbf{x}_1 + h f(t_1, \mathbf{x}_1) \\ &= \begin{bmatrix} 50 \\ -0.98 \end{bmatrix} + 0.1 \begin{bmatrix} -0.98 \\ -9.8 \end{bmatrix} \\ &= \begin{bmatrix} 49.902 \\ -1.96 \end{bmatrix} \end{aligned}$$

Thus $y(0.2) \approx x_{2,1} = 49.902$ and $y'(0.2) \approx x_{2,2} = -1.96$.

(iii) Step-by-step calculation of Euler's method using MATLAB:

```
f = @(t, x) [x(2); -9.8];  
h = 0.1;  
t0 = 0  
x0 = [50; 0]  
  
t1 = t0 + h  
x1 = x0 + h * f(t0, x0)  
  
t2 = t1 + h  
x2 = x1 + h * f(t1, x1)
```

These yield

$$\begin{array}{ll} t_0 = 0 & x_0 = \begin{bmatrix} 50 \\ 0 \end{bmatrix} \\ t_1 = 0.1 & x_1 = \begin{bmatrix} 50 \\ -0.98 \end{bmatrix} \\ t_2 = 0.2 & x_2 = \begin{bmatrix} 49.902 \\ -1.96 \end{bmatrix} \end{array}$$

Thus $y(0.2) \approx 49.902$ and $y'(0.2) \approx -1.96$.

(iv) Step-by-step calculation of Euler's method using Python:

```
def f(t, x): return array([x[1], -9.8])  
h = 0.1  
t0 = 0  
x0 = array([50, 0])  
  
t1 = t0 + h  
x1 = x0 + h * f(t0, x0)  
  
t2 = t1 + h  
x2 = x1 + h * f(t1, x1)
```

The Python results for x_0 , x_1 and x_2 are arrays so will “look like” row vectors.

(v) Solution using `euler.m` (download from Moodle):

```
f = @(t, x) [x(2); -9.8];  
t0 = 0; tmax = 0.2; N = 2;  
x0 = [50; 0];  
  
[t, y] = euler(f, t0, tmax, N, x0)
```

These yield

$$\mathbf{t} = \begin{bmatrix} 0 \\ 0.1 \\ 0.2 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 50 & 0 \\ 50 & -0.98 \\ 49.902 & -1.96 \end{bmatrix}$$

Thus $y(0.2) \approx 49.902$ and $y'(0.2) \approx -1.96$.

(vi) Solution using `euler` from `math2089ode.py` (download from Moodle):

```
from math2089ode import euler  
  
def f(t, x): return array([x[1], -9.8])  
t0 = 0; tmax = 0.2; N = 2;  
x0 = array([50, 0])  
  
t, y = euler(f, t0, tmax, N, x0)
```

The Python result for `t` is an array and so it will “look like” a row vector. The Python result for `y` is a matrix the same as for MATLAB above.

Exercise. Write the second order IVP

$$y'' = 2yt + y', \quad y(1) = 2, \quad y'(1) = 1$$

as a system of first order ODEs. Use Euler's method with step size $h = 0.2$ to estimate $y(1.4)$.

Answer.

(i) Let

Thus we obtain $\mathbf{x}' = \mathbf{f}(t, \mathbf{x})$, with

(ii) Euler's method for this problem is

Hand calculation of two steps with $h = 0.2$:

$$\begin{aligned} t_0 = 1 \quad \mathbf{x}_0 &= \begin{bmatrix} 2 \\ 1 \end{bmatrix} & f(t_0, \mathbf{x}_0) &= \begin{bmatrix} x_{0,2} \\ 2x_{0,1}t_0 + x_{0,2} \end{bmatrix} \\ & & &= \begin{bmatrix} 1 \\ 2 \times 2 \times 1 + 1 \end{bmatrix} \\ & & &= \begin{bmatrix} 1 \\ 5 \end{bmatrix} \\ t_1 = 1.2 \quad \mathbf{x}_1 &= \mathbf{x}_0 + h f(t_0, \mathbf{x}_0) & f(t_1, \mathbf{x}_1) &= \begin{bmatrix} x_{1,2} \\ 2x_{1,1}t_1 + x_{1,2} \end{bmatrix} \\ &= \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 0.2 \begin{bmatrix} 1 \\ 5 \end{bmatrix} & &= \begin{bmatrix} 2 \\ 2 \times 2.2 \times 1.2 + 2 \end{bmatrix} \\ &= \begin{bmatrix} 2.2 \\ 2 \end{bmatrix} & &= \begin{bmatrix} 2 \\ 7.28 \end{bmatrix} \\ t_2 = 1.4 \quad \mathbf{x}_2 &= \mathbf{x}_1 + h f(t_1, \mathbf{x}_1) \\ &= \begin{bmatrix} 2.2 \\ 2 \end{bmatrix} + 0.2 \begin{bmatrix} 2 \\ 7.28 \end{bmatrix} \\ &= \begin{bmatrix} 2.6 \\ 3.456 \end{bmatrix} \end{aligned}$$

Thus

(iii) Step-by-step calculation using MATLAB:

```
f = @(t, x) [x(2); 2 * x(1) * t + x(2)];  
h = 0.2;  
t0 = 1  
x0 = [2; 1]  
  
t1 = t0 + h  
x1 = x0 + h * f(t0, x0)  
  
t2 = t1 + h  
x2 = x1 + h * f(t1, x1)
```

These yield

$$\begin{array}{ll} t_0 = 1 & x_0 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ t_1 = 1.2 & x_1 = \begin{bmatrix} 2.2 \\ 2 \end{bmatrix} \\ t_2 = 1.4 & x_2 = \begin{bmatrix} 2.6 \\ 3.456 \end{bmatrix} \end{array}$$

Thus $y(1.4) \approx 2.6$ and $y'(1.4) \approx 3.456$.

(iv) Step-by-step calculation of Euler's method using Python:

```
def f(t, x): return array([x[1], 2 * x[0] * t + x[1]])  
h = 0.2  
t0 = 1  
x0 = array([2, 1])  
  
t1 = t0 + h  
x1 = x0 + h * f(t0, x0)  
  
t2 = t1 + h  
x2 = x1 + h * f(t1, x1)
```

The Python results for x_0 , x_1 and x_2 are arrays so will “look like” row vectors.

(v) Solution using `euler.m` (download from Moodle):

```
f = @(t, x) [x(2); 2 * x(1) * t + x(2)];  
t0 = 1; tmax = 1.4; N = 2;  
x0 = [2; 1];
```

```
[t, y] = euler(f, t0, tmax, N, x0)
```

These yield

$$\mathbf{t} = \begin{bmatrix} 1 \\ 1.2 \\ 1.4 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 2 & 1 \\ 2.2 & 2 \\ 2.6 & 3.456 \end{bmatrix}$$

Thus $y(1.4) \approx 2.6$ and $y'(1.4) \approx 3.456$.

(vi) Solution using `euler` from `math2089ode.py` (download from Moodle):

```
from math2089ode import euler  
  
def f(t, x): return array([x[1], 2 * x[0] * t + x[1]])  
t0 = 1; tmax = 1.4; N = 2;  
x0 = array([2, 1])  
  
t, y = euler(f, t0, tmax, N, x0)
```

The Python result for `t` is an array and so it will “look like” a row vector. The Python result for `y` is a matrix the same as for MATLAB above.

§6C. Other Methods for Solving IVP.

● Recall common setup...

- ODE $y'(t) = f(t, y)$
- Equally spaced time steps t_n , with step size h
- True solution at t_n is denoted by $y(t_n)$
- Approximation at t_n is denoted by y_n
- Error at t_n is $E_n := y(t_n) - y_n$

● Derivation of Euler's method

$$y_{n+1} = y_n + h f(t_n, y_n)$$

- Via *Taylor approximation of order 1*: since $t_{n+1} = t_n + h$ and $y'(t) = f(t, y)$

$$\begin{aligned} y(t_{n+1}) &= y(t_n + h) = y(t_n) + h y'(t_n) + \frac{h^2}{2} y''(\zeta_n), \quad \zeta_n \in (t_n, t_{n+1}) \\ &= y(t_n) + h f(t_n, y(t_n)) + \frac{h^2}{2} y''(\zeta_n) \\ &\approx y(t_n) + h f(t_n, y(t_n)) \end{aligned}$$

- Via numerical integration using *left-rectangle rule*

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} y'(t) dt = \int_{t_n}^{t_{n+1}} f(t, y) dt \approx h f(t_n, y(t_n))$$

- Error of Euler's method at final step t_N : $E_N = \mathcal{O}(h)$ order 1
~ slow convergence ~

Notes:

- (i) Local truncation error = $\mathcal{O}(h^2)$ in each step, compounding to
Global truncation error = $N \times \mathcal{O}(h^2) = \mathcal{O}(h)$
- (ii) Local roundoff error = $\mathcal{O}(\varepsilon)$ in each step, compounding to
Combined roundoff error = $N \times \mathcal{O}(\varepsilon) = \mathcal{O}(\frac{\varepsilon}{h})$

● Other numerical methods for solving IVP (no need to memorize formulas)

● Classification

- **One-step** methods: y_{n+1} is determined solely from a knowledge of y_n
- **Multi-step** methods: y_{n+1} is determined from a number of previous steps $y_n, y_{n-1}, y_{n-2}, y_{n-3}$, etc., thus needs more starting values
- **Explicit** methods: y_{n+1} can be obtained from a linear equation
- **Implicit** methods: y_{n+1} requires the solution of a nonlinear equation (e.g. by fixed point iteration) $\sim$ **more stable for "stiff" problems** $\sim$

- **Taylor method** of order 2 (also higher): $E_N = \mathcal{O}(h^2)$, one-step, explicit

$$y_{n+1} = y_n + h f(t_n, y_n) + \frac{h^2}{2} \frac{\partial f(t, y)}{\partial t} \Big|_{t=t_n, y=y_n}$$

- **Implicit Euler's method**: $E_N = \mathcal{O}(h)$, one-step, implicit

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$$

- **Trapezoidal method**: $E_N = \mathcal{O}(h^2)$, one-step, implicit

$$y_{n+1} = y_n + \frac{h}{2} [f(t_n, y_n) + f(t_{n+1}, y_{n+1})]$$

- A 2-stage **Runge-Kutta method** (RK2): $E_N = \mathcal{O}(h^2)$, one-step, explicit

$$\begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + \frac{2}{3}h, y_n + \frac{2}{3}hk_1) \\ y_{n+1} &= y_n + \frac{h}{4} [k_1 + 3k_2] \end{aligned}$$

- A 4-stage **Runge-Kutta method** (RK4): $E_N = \mathcal{O}(h^4)$, one-step, explicit

$$\begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1) \\ k_3 &= f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_2) \\ k_4 &= f(t_n + h, y_n + hk_3) \\ y_{n+1} &= y_n + \frac{h}{6} [k_1 + 2k_2 + 2k_3 + k_4] \end{aligned}$$

- An **Adams-Bashforth method** (AB4): $E_N = \mathcal{O}(h^4)$, 4-step, explicit

$$y_{n+1} = y_n + \frac{h}{24} [55f(t_n, y_n) - 59f(t_{n-1}, y_{n-1}) + 37f(t_{n-2}, y_{n-2}) - 9f(t_{n-3}, y_{n-3})]$$

- An **Adams-Moulton method** (AM4): $E_N = \mathcal{O}(h^4)$, 3-step, implicit

$$y_{n+1} = y_n + \frac{h}{24} [9f(t_{n+1}, y_{n+1}) + 19f(t_n, y_n) - 5f(t_{n-1}, y_{n-1}) + f(t_{n-2}, y_{n-2})]$$

● **Predictor-corrector methods:** combine an explicit method (predictor) with an implicit method (corrector), avoid having to solve a nonlinear equation

● **Heun's method** (Euler's method + trapezoidal method): $E_N = O(h^2)$, one-step, explicit

$$\text{Predictor: } \bar{y}_{n+1} = y_n + hf(t_n, y_n)$$

$$\text{Corrector: } y_{n+1} = y_n + \frac{h}{2} [f(t_n, y_n) + f(t_{n+1}, \bar{y}_{n+1})]$$

$$\text{or equivalently, } y_{n+1} = y_n + \frac{h}{2} [f(t_n, y_n) + f(t_{n+1}, y_n + hf(t_n, y_n))]$$

● **PC4 (AB4 + AM4):** $E_N = O(h^4)$, 4-step, explicit

$$\bar{y}_{n+1} = y_n + \frac{h}{24} [55f(t_n, y_n) - 59f(t_{n-1}, y_{n-1}) + 37f(t_{n-2}, y_{n-2}) - 9f(t_{n-3}, y_{n-3})]$$

$$y_{n+1} = y_n + \frac{h}{24} [9f(t_{n+1}, \bar{y}_{n+1}) + 19f(t_n, y_n) - 5f(t_{n-1}, y_{n-1}) + f(t_{n-2}, y_{n-2})]$$

Example. Consider the first order IVP

$$y' = 1 + \frac{y}{t}, \quad t \in [1, 3], \quad y(1) = 2.$$

Use the 2-stage Runge-Kutta method (RK2) on Page 20 and Heun's method with step size $h = 0.2$ to estimate $y(1.2)$ and $y(1.4)$.

Answer. For this problem we have $f(t, y) = 1 + \frac{y}{t}$.

(i) Hand calculation for one step of RK2 with $h = 0.2$:

$$t_0 = 1 \quad y_0 = 2$$

$$k_1 = f(t_0, y_0)$$

$$= 1 + \frac{y_0}{t_0}$$

$$= 1 + \frac{2}{1} = 3$$

$$k_2 = f(t_0 + \frac{2}{3}h, y_0 + \frac{2}{3}h k_1)$$

$$= 1 + \frac{y_0 + \frac{2}{3}h k_1}{t_0 + \frac{2}{3}h}$$

$$= 1 + \frac{2 + \frac{2}{3} \times 0.2 \times 3}{1 + \frac{2}{3} \times 0.2} \approx 3.1176$$

$$\begin{aligned} t_1 = 1.2 \quad y_1 &= y_0 + \frac{h}{4}(k_1 + 3k_2) \\ &= 2 + \frac{0.2}{4}(3 + 3 \times 3.1176) \\ &\approx 2.6176 \end{aligned}$$

Thus $y(1.2) \approx 2.6176$.

(ii) Step-by-step calculation of RK2 using MATLAB :

```
f = @(t, y) 1 + y ./ t;  
h = 0.2;  
t0 = 1  
y0 = 2  
  
t1 = t0 + h  
k1 = f(t0, y0)  
k2 = f(t0 + (2/3)*h, y0 + (2/3)*h * k1)  
y1 = y0 + h/4 * (k1 + 3*k2)  
  
t2 = t1 + h  
k1 = f(t1, y1)  
k2 = f(t1 + (2/3)*h, y1 + (2/3)*h * k1)  
y2 = y1 + h/4 * (k1 + 3*k2)
```

These yield

t0 = 1				y0 = 2
t1 = 1.2	k1 = 3	k2 ≈ 3.1176		y1 ≈ 2.6176
t2 = 1.4	k1 ≈ 3.1814	k2 ≈ 3.2814		y2 ≈ 3.2689

Thus $y(1.2) \approx 2.6176$ and $y(1.4) \approx 3.2689$.

(iii) Step-by-step calculation of RK2 using Python :

```
def f(t, y): return 1 + y / t  
h = 0.2  
t0 = 1  
y0 = 2  
  
t1 = t0 + h  
k1 = f(t0, y0)  
k2 = f(t0 + (2/3)*h, y0 + (2/3)*h * k1)  
y1 = y0 + h/4 * (k1 + 3*k2)  
  
t2 = t1 + h  
k1 = f(t1, y1)  
k2 = f(t1 + (2/3)*h, y1 + (2/3)*h * k1)  
y2 = y1 + h/4 * (k1 + 3*k2)
```

(iv) Step-by-step calculation of Heun's method using MATLAB:

```
f = @(t, y) 1 + y ./ t;  
h = 0.2;  
t0 = 1  
y0 = 2  
  
t1 = t0 + h  
f0 = f(t0, y0)  
y1p = y0 + h * f0  
y1 = y0 + h/2 * ( f0 + f(t1, y1p) )  
  
t2 = t1 + h  
f1 = f(t1, y1)  
y2p = y1 + h * f1  
y2 = y1 + h/2 * ( f1 + f(t2, y2p) )
```

These yield

t0 = 1			y0 = 2
t1 = 1.2	f0 = 3	y1p = 2.6	y1 ≈ 2.6167
t2 = 1.4	f1 ≈ 3.1806	y2p ≈ 3.2528	y2 ≈ 3.2671

Thus $y(1.2) \approx 2.6167$ and $y(1.4) \approx 3.2671$.

(v) Step-by-step calculation of Heun's method using Python:

```
def f(t, y): return 1 + y / t  
h = 0.2  
t0 = 1  
y0 = 2  
  
t1 = t0 + h  
f0 = f(t0, y0)  
y1p = y0 + h * f0  
y1 = y0 + h/2 * ( f0 + f(t1, y1p) )  
  
t2 = t1 + h  
f1 = f(t1, y1)  
y2p = y1 + h * f1  
y2 = y1 + h/2 * ( f1 + f(t2, y2p) )
```

Example. MATLAB `heun.m` (download from Moodle) provides a function to solve a system of first order ODEs by Heun's method:

```
function [t, y] = heun(f, t0, tmax, N, y0)
```

It differs from `euler.m` on Page 9 only within the loop:

```
for n = 1:N
    fn = f(t(n), y(:,n));
    ynp = y(:,n) + h * fn; % predictor
    y(:,n+1) = y(:,n) + (h/2) * ( fn + f(t(n+1), ynp) ); % corrector
end
```

Python `math2089ode.py` (download from Moodle) provides an equivalent function:

```
def heun(f, t0, tmax, N, y0):
```

It differs from the function `euler` on Page 9 only within the loop:

```
for n in range(N):
    fn = f(t[n], y[n,:])
    ynp = y[n,:] + h * fn # predictor
    y[n+1,:] = y[n,:] + (h/2) * ( fn + f(t[n+1], ynp) ) # corrector
```

Example. MATLAB `rk4.m` (download from Moodle) provides a function to solve a system of first order ODEs by the 4-stage Runge-Kutta method on Page 20.

```
function [t, y] = rk4(f, t0, tmax, N, y0)
```

It differs from `euler.m` on Page 9 only within the loop:

```
for n = 1:N
    k1 = f(t(n), y(:,n));
    k2 = f(t(n) + h/2, y(:,n) + (h/2) * k1);
    k3 = f(t(n) + h/2, y(:,n) + (h/2) * k2);
    k4 = f(t(n) + h, y(:,n) + h * k3);
    y(:,n+1) = y(:,n) + (h/6) * ( k1 + 2*k2 + 2*k3 + k4 ); % RK4
end
```

Python `math2089ode.py` (download from Moodle) provides an equivalent function:

```
def rk4(f, t0, tmax, N, y0):
```

It differs from the function `euler` on Page 9 only within the loop:

```
for n in range(N):
    k1 = f(t[n], y[n,:])
    k2 = f(t[n] + h/2, y[n,:] + (h/2) * k1)
    k3 = f(t[n] + h/2, y[n,:] + (h/2) * k2)
    k4 = f(t[n] + h, y[n,:] + h * k3)
    y[n+1,:] = y[n,:] + (h/6) * ( k1 + 2*k2 + 2*k3 + k4 ) # RK4
```

Exercise. Write the second order IVP

$$y'' - y' e^{-y} - 2ty = t^2, \quad t \in [1, 2], \quad y(1) = 0, \quad y'(1) = 1$$

as a system of first order ODEs. Use the 2-stage Runge-Kutta method (RK2) on Page 20 with step size $h = 0.1$ to estimate $y(1.2)$ and $y'(1.2)$.

Answer.

(i) Let

Thus we obtain $\mathbf{x}' = \mathbf{f}(t, \mathbf{x})$, with

(ii) Step-by-step calculation of RK2 using MATLAB:

```
f = @(t, x) [x(2); x(2)*exp(-x(1)) + 2*t*x(1) + t^2];
h = 0.1;
t0 = 1
x0 = [0; 1]

t1 = t0 + h
k1 = f(t0, x0)
k2 = f(t0 + (2/3)*h, x0 + (2/3)*h * k1)
x1 = x0 + h/4 * (k1 + 3*k2)

t2 = t1 + h
k1 = f(t1, x1)
k2 = f(t1 + (2/3)*h, x1 + (2/3)*h * k1)
x2 = x1 + h/4 * (k1 + 3*k2)
```

These yield

$$\begin{array}{llll} t_0 = 1 & & & \mathbf{x}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ t_1 = 1.1 & \mathbf{k}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} & \mathbf{k}_2 \approx \begin{bmatrix} 1.1333 \\ 2.3402 \end{bmatrix} & \mathbf{x}_1 \approx \begin{bmatrix} 0.1100 \\ 1.2255 \end{bmatrix} \\ t_2 = 1.2 & \mathbf{k}_1 \approx \begin{bmatrix} 1.2255 \\ 2.5499 \end{bmatrix} & \mathbf{k}_2 \approx \begin{bmatrix} 1.3955 \\ 2.9605 \end{bmatrix} & \mathbf{x}_2 \approx \begin{bmatrix} 0.2453 \\ 1.5113 \end{bmatrix} \end{array}$$

Thus

(iii) Step-by-step calculation of RK2 using Python:

```
from numpy import array, exp

def f(t, x):
    return array([x[1], x[1]*exp(-x[0]) + 2*t*x[0] + t**2])
h = 0.1
t0 = 1
x0 = array([0, 1])

t1 = t0 + h
k1 = f(t0, x0)
k2 = f(t0 + (2/3)*h, x0 + (2/3)*h * k1)
x1 = x0 + h/4 * (k1 + 3*k2)

t2 = t1 + h
k1 = f(t1, x1)
k2 = f(t1 + (2/3)*h, x1 + (2/3)*h * k1)
x2 = x1 + h/4 * (k1 + 3*k2)
```

The Python results for x_0 , x_1 , x_2 , k_1 and k_2 are arrays so will “look like” row vectors.

Example. For the same IVP as in the previous exercise, use MATLAB `rk4.m` or Python `rk4` from `math2089ode.py` with step size $h = 0.1$ to estimate $y(2)$ and $y'(2)$.

Answer. In MATLAB we do

```
f = @(t, x) [x(2); x(2)*exp(-x(1)) + 2*t*x(1) + t^2];
t0 = 1;
tmax = 2;
N = 10;
x0 = [0; 1];

[t, y] = rk4(f, t0, tmax, N, x0); % same syntax for euler and heun

format long
y_end = y(end,1)
dy_end = y(end,2)
```

to get the output

```
y_end =
    3.432954434760687
dy_end =
    8.288590399102867
```

In Python we do

```
from numpy import array, exp
from math2089ode import *

def f(t, x):
    return array([x[1], x[1]*exp(-x[0]) + 2*t*x[0] + t**2])
t0 = 1
tmax = 2
N = 10
x0 = array([0, 1])

t, y = rk4(f, t0, tmax, N, x0) # same syntax for euler and heun

print(' y_end =', y[-1,0])
print('dy_end =', y[-1,1])
```

to get the output

```
y_end = 3.432892699628995
dy_end = 8.28852340076822
```

§6D. Stiff problems and BVP with Shooting Method.

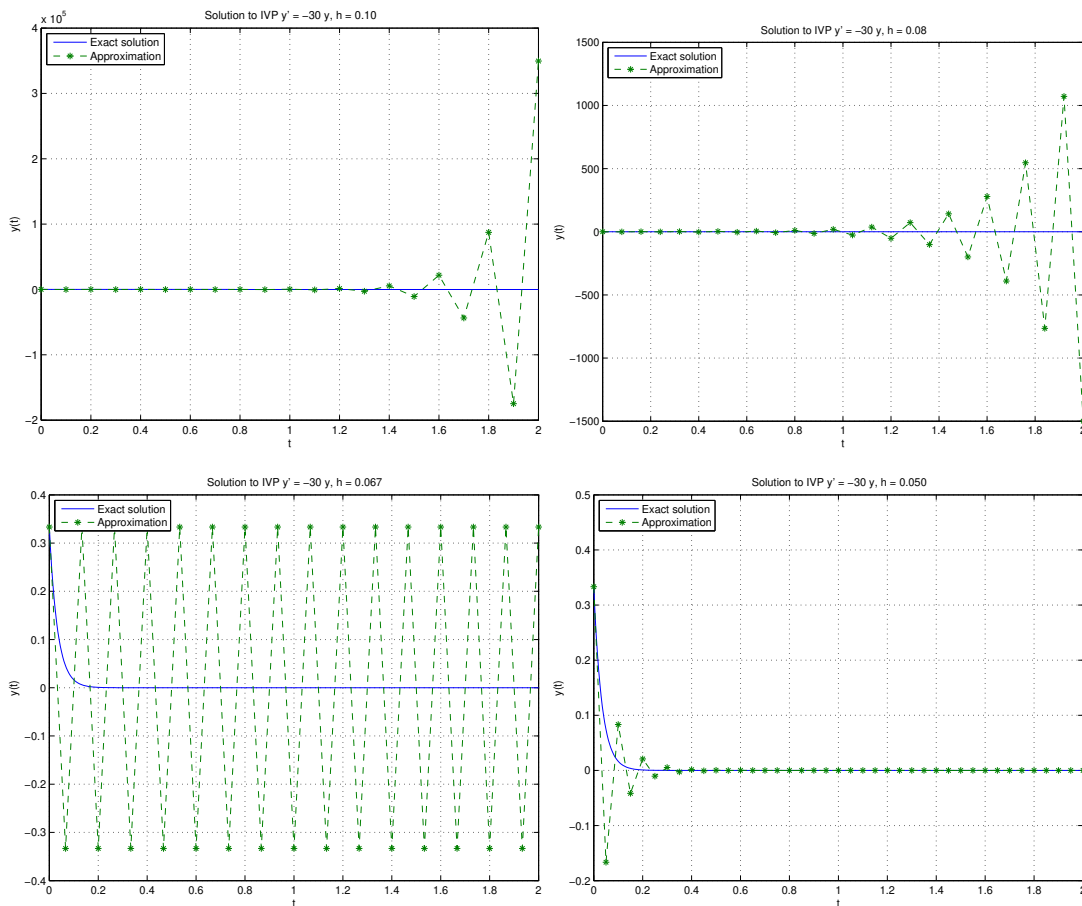
- A **stiff** ODE is an equation for which some numerical methods require a very small step size to avoid instability
- e.g. $y' = cy$ with $c < 0$
- Explicit methods only work if step size is small
- Implicit methods are more stable for stiff problems

Example. Consider the IVP: $y' = -30y$ for $t \in [0, 2]$, with $y_0 = 1/3$. The true solution is $y(t) = \frac{1}{3} e^{-30t} \rightarrow 0$ as $t \rightarrow \infty$.

Euler's method:

$$y_{n+1} = y_n + h f(t_n, y_n) = y_n + h(-30y_n) = (1 - 30h)y_n = \dots = (1 - 30h)^{n+1}y_0,$$

which converges if and only if $|1 - 30h| < 1 \iff h < \frac{1}{15} \approx 0.067$.



Implicit Euler's method:

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1}) = y_n + h(-30y_{n+1}) \implies (1 + 30h)y_{n+1} = y_n$$

$$\implies y_{n+1} = \left(\frac{1}{1+30h}\right)y_n = \dots = \left(\frac{1}{1+30h}\right)^{n+1}y_0,$$

which converges for all h since $\frac{1}{1+30h} < 1$ for all h .

- MATLAB `ode45` (built-in ODE solver) uses the difference between explicit Runge-Kutta methods of orders 4 and 5 to estimate the error, and adjust the step size to keep error estimate within the desired tolerance.
- Python `solve_ivp` from `scipy.integrate` implements a number of different methods. The default method is 'RK45' which is essentially the same as `ode45`.
- MATLAB `ode45` has a 'Refine' option which defaults to 4, meaning that for every computed point it adds 3 additional interpolated points. With the same tolerance and approximately the same work, `ode45` will return roughly 4 times more points than `solve_ivp`, giving a smoother looking plot.

Example.

```
f = @(t, y) -30*y;
t0 = 0;
tmax = 2;
y0 = 1/3; % m = 1 is the dimension of the IVP

% N = 20; % try N = 20, 25, 30, 40 (h = 0.1, 0.08, 1/15, 0.05)
% [t, y] = euler(f, t0, tmax, N, y0); % or heun or rk4

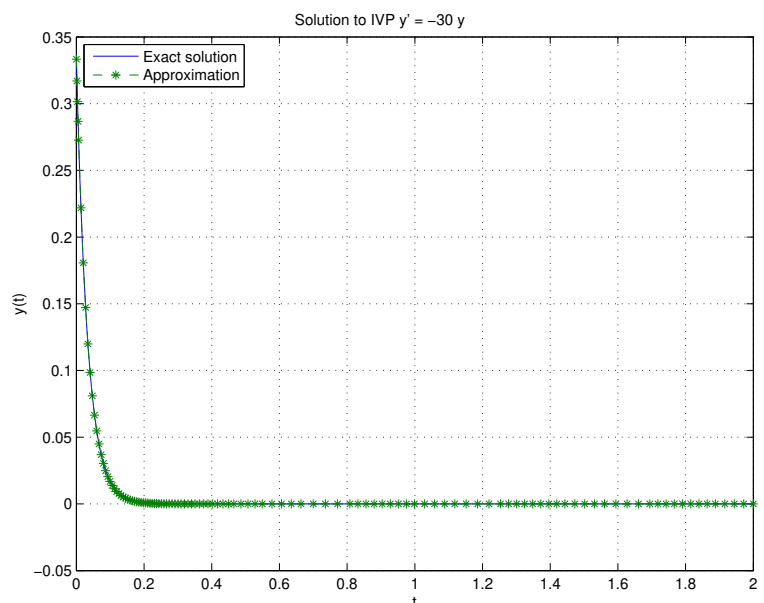
opt = odeset('RelTol', 1e-7, 'AbsTol', 1e-10);
% default tolerance values are 1e-3 and 1e-6

[t, y] = ode45(f, [t0, tmax], y0, opt);
plot(t, y, '*-'); % t is a vector of size N_steps by 1
% y is a matrix of size N_steps by m

N_steps = length(t)
y_end = y(end)
```

We get the output

```
N_steps =
    397
y_end =
    4.863323397534807e-12
```



Example.

```
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def f(t, y): return -30*y
t0 = 0
tmax = 2
y0 = 1/3 # m = 1 is the dimension of the IVP

# N = 20 # try N = 20, 25, 30, 40 (h = 0.1, 0.08, 1/15, 0.05)
# t, y = euler(f, t0, tmax, N, y0) # or heun or rk4

output = solve_ivp(f, [t0, tmax], [y0], rtol = 1e-7, atol = 1e-10)
# [y0] turns it into an array as required
# default tolerance values are rtol = 1e-3, atol = 1e-6

t = output.t # t is an array of size N_steps
y = output.y.T # y is a matrix of size N_steps by m
plt.plot(t, y, '*-')

print('N_steps =', len(t))
print('y_end =', y[-1])
```

We get the output

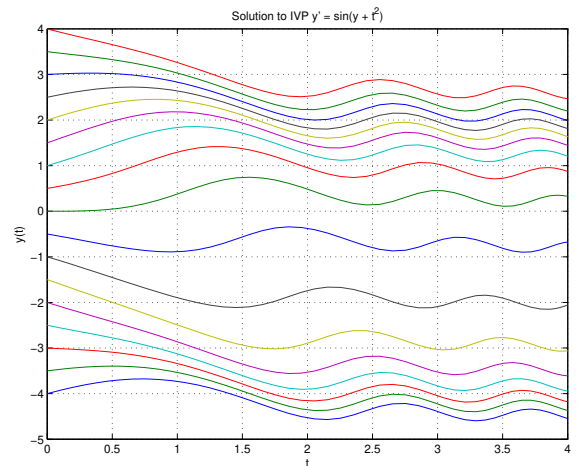
```
N_steps = 88
y_end = [1.90888402e-11]
```

Example.

```
f = @(t, y) sin(y + t.^2);
t0 = 0;
tmax = 4;           % interpret as a system of 17 independent IVPs
y0 = [-4:0.5:4]';   % m = 17 different initial values

opt = odeset('RelTol', 1e-7, 'AbsTol', 1e-10);
[t, y] = ode45(f, [t0, tmax], y0, opt);
plot(t, y);
% t is a vector of size N_steps by 1
% y is a matrix of size N_steps by m

N_steps = length(t)
y_end = y(end, [1,9,17])
```



We get the output

```
N_steps =
    245
y_end =
   -4.547941930981085    0.325473457093263    2.466977996038012
```

Example.

```
from numpy import sin, linspace
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def f(t, y): return sin(y + t**2)
t0 = 0
tmax = 4                                # a system of 17 independent IVPs
y0 = linspace(-4, 4, 17)               # m = 17 different initial values

output = solve_ivp(f, [t0, tmax], y0, rtol = 1e-7, atol = 1e-10)
t = output.t                           # t is an array of size N_steps
y = output.y.T                         # y is a matrix of size N_steps by m
plt.plot(t, y)

print('N_steps =', len(t))
print('y_end =', y[-1, [0,8,16]])
```

We get the output

```
N_steps = 46
y_end = [-4.54794197  0.32547344  2.46697796]
```

Example.

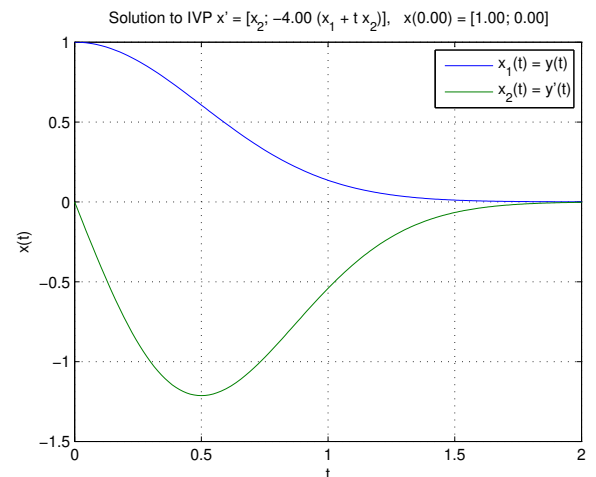
```
f = @(t, x) [x(2); -4*(x(1)+t.*x(2))];
t0 = 0;
tmax = 2;
x0 = [1;0]; % m = 2 is the dimension of the IVP
```

```
opt = odeset('RelTol', 1e-7, 'AbsTol', 1e-10);
[t, y] = ode45(f, [t0, tmax], x0, opt);
plot(t, y);
% t is a vector of size N_steps by 1
% y is a matrix of size N_steps by m
```

```
N_steps = length(t)
y_end = y(end, :)
```

We get the output

```
N_steps =
    241
y_end =
    0.000335462661148   -0.002683701289183
```



Example.

```
from numpy import array
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def f(t, x): return array([x[1], -4*(x[0]+t*x[1])])
t0 = 0
tmax = 2
x0 = array([1, 0]) # m = 2 is the dimension of the IVP

output = solve_ivp(f, [t0, tmax], x0, rtol = 1e-7, atol = 1e-10)
t = output.t # t is an array of size N_steps
y = output.y.T # y is a matrix of size N_steps by m
plt.plot(t, y)

print('N_steps =', len(t))
print('y_end =', y[-1, :])
```

We get the output

```
N_steps = 51
y_end = [ 0.00033546 -0.0026837 ]
```

● Two-point *Boundary Value Problem* (BVP)

$$y'' = f(t, y, y') \quad \text{for } t \in [a, b], \quad \text{with } y(a) = y_a \text{ and } y(b) = y_b$$

- Given function $f(t, y, y')$
- Given interval $[a, b]$
- Given *boundary values* y_a and y_b
- Unknown function $y(t)$ to be found

● *Shooting methods*

- The above BVP is related to an IVP with a parameter η :

$$y'' = f(t, y, y') \quad \text{for } t \in [a, b], \quad \text{with } y(a) = y_a \text{ and } y'(a) = \eta,$$

subject to the condition that its solution $y(t; \eta)$ satisfies $y(b; \eta) = y_b$.

- Main idea: *solve the nonlinear equation $y(b; \eta) - y_b = 0$ to find η* , using iterative methods (bisection, fixed point iteration, Newton, secant)
- Informally, this can be done by trial-and-error

Example. Consider the BVP

$$y'' - \frac{y'}{t} + \frac{y}{t^2} = 0, \quad y(1) = 2, \quad y(2) = 6.$$

- Verify that the exact solution is $y(t) = 2t + \frac{t \ln(t)}{\ln(2)}$.

- The BVP is related to the IVP with parameter η :

$$y'' - \frac{y'}{t} + \frac{y}{t^2} = 0, \quad y(1) = 2, \quad y'(1) = \eta,$$

subject to the condition that its solution $y(t; \eta)$ satisfies $y(2; \eta) = 6$.

- Convert the IVP into a system of first order ODEs

- Shooting method by trial-and-error with MATLAB `ode45`

(i) Solve this IVP with the initial guess of $\eta_1 = 1$:

```
f = @(t, x) [x(2); x(2)./t - x(1)./(t.^2)];  
a = 1; b = 2; ya = 2; yb = 6;  
opt = odeset('RelTol', 1e-7, 'AbsTol', 1e-10);  
format long  
  
eta1 = 1;  
x0 = [ya; eta1]; % m = 2 is the dimension of the IVP  
[t, y] = ode45(f, [a, b], x0, opt);  
% t is a vector of size N_steps by 1  
% y is a matrix of size N_steps by m  
yb1 = y(end, 1)
```

The output is

```
yb1 =  
2.613705635057605
```

But the correct value is $y(2) = 6$.

(ii) Next we take another guess of $\eta_2 = 5$ and shoot again:

```
eta2 = 5;  
x0 = [ya; eta2];  
[t, y] = ode45(f, [a, b], x0, opt);  
yb2 = y(end, 1)
```

The output is

```
yb2 =  
8.158883092619341
```

But the correct value is $y(2) = 6$.

(iii) We conclude that the correct value of η is between $\eta_1 = 1$ and $\eta_2 = 5$. We can take another guess in this range and shoot again...

Alternatively, we can take the value of η obtained by linear interpolation:

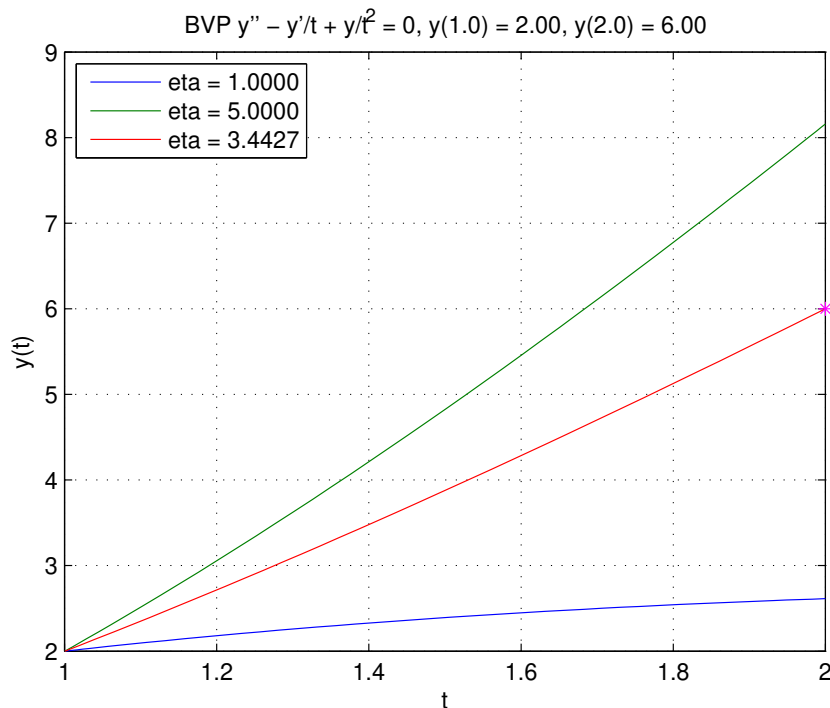
$$\frac{\text{eta2} - \text{eta1}}{y_{b2} - y_{b1}} = \frac{\eta - \text{eta1}}{y_b - y_{b1}} \implies \eta = \frac{\text{eta2} - \text{eta1}}{y_{b2} - y_{b1}}(y_b - y_{b1}) + \text{eta1}$$

```
eta3 = (eta2 - eta1) / (yb2 - yb1) * (yb - yb1) + eta1
x0 = [ya; eta3];
[t, y] = ode45(f, [a, b], x0, opt);
yb3 = y(end, 1)
```

The output is

```
eta3 =
    3.442695037883515
yb3 =
    6.000000001012033
```

So we are satisfied that the solution for this IVP is also the solution for the BVP.



- (iv) If the value of η obtained from interpolation is not good enough, then the process can be repeated until the solution is sufficiently close to the given boundary condition $y(2) = 6$.
- (v) The trial-and-error approach used in this example only illustrates the idea behind the shooting methods. The real shooting methods are formulated using methods from Topic 4 for nonlinear equations.

• Shooting method by trial-and-error with Python `scipy.integrate.solve_ivp`

(i) Solve this IVP with the initial guess of $\eta_1 = 1$:

```
from numpy import array
from scipy.integrate import solve_ivp
def f(t, x): return array([x[1], x[1]/t - x[0]/(t**2)])
a = 1; b = 2; ya = 2; yb = 6

eta1 = 1
x0 = array([ya, eta1]) # m = 2 is the dimension of the IVP
output = solve_ivp(f, [a, b], x0, rtol = 1e-7, atol = 1e-10)
t = output.t           # t is an array of size N_steps
y = output.y.T         # y is a matrix of size N_steps by m
yb1 = y[-1, 0]
print('yb1 =', yb1)
```

The output is

```
yb1 = 2.6137055776159355
```

But the correct value is $y(2) = 6$.

(ii) Next we take another guess of $\eta_2 = 5$ and shoot again:

```
eta2 = 5
x0 = array([ya, eta2])
output = solve_ivp(f, [a, b], x0, rtol = 1e-7, atol = 1e-10)
y = output.y.T
yb2 = y[-1, 0]
print('yb2 =', yb2)
```

The output is

```
yb2 = 8.158883203289227
```

But the correct value is $y(2) = 6$.

(iii) We take the value of η obtained by linear interpolation and shoot again:

```
eta3 = (eta2 - eta1) / (yb2 - yb1) * (yb - yb1) + eta1
x0 = array([ya, eta3])
output = solve_ivp(f, [a, b], x0, rtol = 1e-7, atol = 1e-10)
y = output.y.T
yb3 = y[-1, 0]
print('eta3 =', eta3)
print('yb3 =', yb3)
```

The output is

```
eta3 = 3.442695005264437
yb3 = 6.00000004257056
```

We are satisfied that the solution for this IVP also solves the BVP.